

**AIM: Cryptography in Blockchain, Merkle root Tree Hash**

**Lab Objectives:** To explore Blockchain concepts.

**Lab Outcomes (LO):** Creating Cryptographic Hash using Merkle Tree (LO1)

**Task to be performed :**

1. Make a copy of this [Google Colab Notebook](#)
2. Try to solve the errors in each of the 4 Programs
3. In the 4th Program - Constructing a Merkle Tree Root Hash, modify the code as follows:

- Update the transactions list with valid entries.

eg : transactions = ['A -> B : \$10', 'B -> C : \$5', 'C -> A : \$2']

Sample Transactions to be considered

T1 : Alice → Bob : \$200;      T2 : Bob → Dave : \$500;      T3 : Dave → Eve : \$100

T4 : Eve → Alice : \$300;      T5 : Roo → Bob : \$50

- Hash the transactions before combining them in the for loop
- Print all the intermediate hash during the construction of the Merkle Tree Root Hash

**Tools & Libraries used :**

- Python Libraries: **hashlib**

**Instructions :** (Prepare for viva for the following topics)

1. Cryptographic Hash functions in Blockchain
2. What is a Merkle Tree?
3. What is a Cryptographic Puzzle and explain the Golden Nonce
4. How does a Merkle Tree work?
5. Benefits of Merkle Tree
6. Use cases of Merkle Tree

**Outcome :**

1. Understood the concept of hashing, Cryptographic Puzzle to find nonce, Merkle Tree and its relevance.
2. Implemented programs
  - To find the Golden Nonce while solving the Cryptographic Puzzle
  - To construct a Merkle Tree for the given transactions
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**  
(As per the instructions, submit a hard copy of the same).

**Aim:** Cryptography in Blockchain, Merkle root Tree Hash

**Tasks Performed:**

**1. Hash Generation using SHA-256:**

- o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

**2. Target Hash Generation with Nonce:**

- o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

**3. Proof-of-Work Puzzle Solving:**

- o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

**4. Merkle Tree Construction:**

- o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

**Program and Output:**

Program 1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib

def create_hash(string):
    hash_object = hashlib.sha256()
    hash_object.update(string.encode('utf-8'))
    hash_string = hash_object.hexdigest()
    return hash_string

input_string = input("Enter a string: ")
hash_result = create_hash(input_string)

print("Hash:", hash_result)

Enter a string: diya
Hash: da9660c38bcd7f409ec94a6c749af6c4cb1af22699071fc2d32a2d99acaeaf66
```

Program 2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

Enter a string: diya  
Enter the nonce: 52  
Hash Code: 902286add01e5e5f3660bdbfbada62a7d3bf12b22cf7ee31d9f4738e92ad0de2

Program 3: Python code for Solving Puzzle for leading zeros with expected nonce and given string 3 Error

```

import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash starts with required zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = input("Enter the input string: ")
num_zeros = int(input("Enter the number of leading zeros: "))

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the result
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

```

Enter the input string: diya
Enter the number of leading zeros: 6
Hash: 0000007f40022fbc83d1774fffd878fbf3af2b546a1ba263b8cdcd01c15505c76
Input String: diya
Leading Zeros: 6
Expected Nonce: 44736109

```

#### Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def build_merkle_tree(transactions): # Error 1

    if len(transactions) == 0:
        return None

    # Step 1: Hash all transactions before combining
    transactions = [
        hashlib.sha256(transaction.encode('utf-8')).hexdigest()
        for transaction in transactions
    ]

    print("Initial Transaction Hashes:")
    for i, transaction_hash in enumerate(transactions):
        print(f"T{i+1}: {transaction_hash}")

    print("\nMerkle Tree Construction:")

    level = 1

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:

        # If odd number of hashes, duplicate the last hash
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        for i in range(0, len(transactions), 2):

            # Combine two hashes
            combined = transactions[i] + transactions[i + 1]
```



```
# Combine two hashes
combined = transactions[i] + transactions[i + 1]

# Hash the combined value
hash_combined = hashlib.sha256(
    combined.encode('utf-8')
).hexdigest()
new_transactions.append(hash_combined)          # Error 2

print(
    f"Hash {i//2 + 1}: "
    f"{hash_combined}"
)

transactions = new_transactions

print()

level += 1

return transactions[0]

# Example usage
transactions = [
    "Diya -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Diya : $300",
    "Roo -> Bob : $50"
]

merkle_root = build_merkle_tree(transactions)

print("Merkle Root:", merkle_root)              # Error 3
```

... Initial Transaction Hashes:

T1: 8f02c0f14383c3870d151f0c56e9455890c50ffa6cefd866893a422d60bd88b9  
T2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46  
T3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da  
T4: 7eed63623b37cc7aa55fcbba92234397493941ae12ee6d25e4ee26f9bc0765b8  
T5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Merkle Tree Construction:

Hash 1: c74482fffc64877b3f1fd0129135a2cd4d0d53da9e9441460d8fc0224b093f962  
Hash 2: 9a9a42f2546ca5a0852f69201e4c506765dfb547d62e8572394ade860f2f39c8  
Hash 3: 314ca746b3dfe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Hash 1: d7ea77ef21c4e27776db578b076b9b520bcd6f60022b41735192c17b9567907a3  
Hash 2: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Hash 1: 919b070070f3678502ca6a2f350405daaf8e5d9bc58341823aedfca2eba1fcbf

Merkle Root: 919b070070f3678502ca6a2f350405daaf8e5d9bc58341823aedfca2eba1fcbf

## Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.